



SCHOOL OF COMPUTER TECHNOLOGY

AASD 4008 Big Data Tools and Technologies II

What is Snowflake Platform?

Snowflake Data Cloud Platform revolutionizes **data warehousing**, providing a **cloud-native** solution for businesses seeking unmatched **performance** and **simplicity**. In this presentation, we delve into the key architectural aspects, business applications, and code examples that showcase the platform's versatility.



Continued...

Snowflake, a **fully-managed** cloud data platform, revolutionizes data warehousing. Its architecture is founded on a unique multi-cluster, shared data model, separating **storage** and **compute** for unprecedented scalability and performance. Built natively for the cloud, Snowflake supports **AWS**, **Azure**, and **GCP**.



Continued...

Launched in October 2014 after two years in stealth mode, the Snowflake database was initially designed to disrupt the data warehouse marketplace.

Over time, the cloud data platform was extended to handle a range of workloads, including Data Warehousing, Data Engineering, Data Lakes, Data Science and recently Marketplace

The diagram below illustrates the key components of the Snowflake Data Cloud, which is deployed across all three major cloud providers, AWS, Google and Azure.



What makes up the Snowflake Platform ?



These are the foundation for Snowflake's cloud data platform:

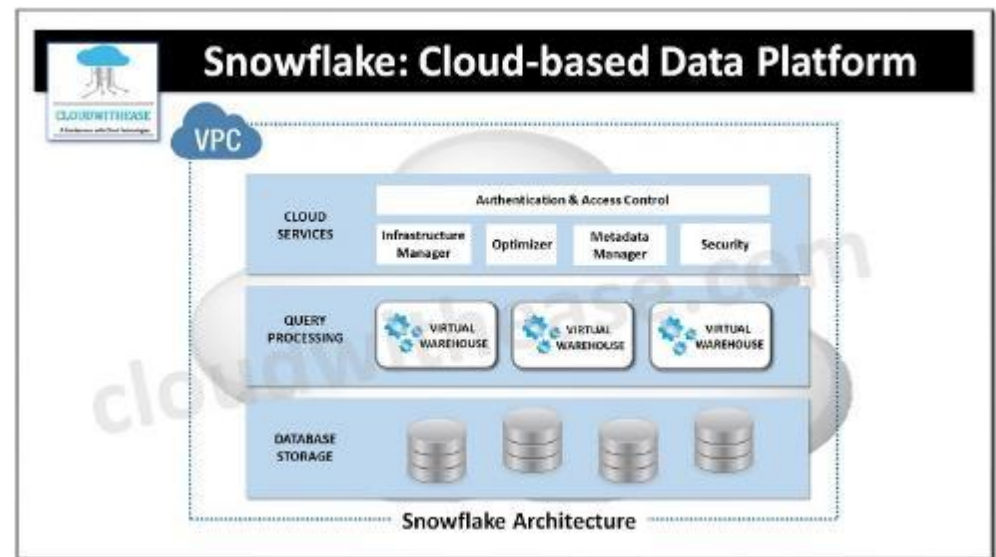
Cloud services. Snowflake uses **ANSI SQL** for cloud services empowering users to optimize their data and manage their infrastructure. Snowflake handles the security and encryption of stored data. They maintain robust data warehousing certifications such as **PCI DSS** and **HIPAA**. Services include authentication, infrastructure management, query parsing and optimization, metadata management, and access control.

Query processing. The compute layer of Snowflake is made up of **virtual cloud data warehouses** that let you analyze data through requests. **Each Snowflake virtual warehouse is an independent cluster** and they do not compete for computing resources nor affect the performance of each other — which means workload concurrency is never a problem.

Database storage. A Snowflake database is where an organization's uploaded **structured** and **semi structured** data sets are held for processing and analysis. Snowflake automatically manages all parts of the data storage process, including organization, structure, metadata, file size, compression, and statistics.

Architecture Overview

Snowflake's architecture employs a multi-cluster, shared data approach. Each cluster handles specific tasks concurrently, ensuring optimal parallel processing. The separation of storage and compute allows independent scaling, providing instant adaptability to changing workloads and minimizing costs.



Business Use Cases



From traditional analytics to advanced use cases, Snowflake excels. It serves as a central hub for **data warehousing**, enables **real-time analytics**, and empowers **machine learning** applications. Its **structured** and **semi-structured** data support caters to a wide array of business needs.

Data Sharing and Collaboration



Snowflake stands out in data sharing and collaboration. It facilitates secure, **real-time cross-organizational data sharing**, controlled by **intricate access policies**. This capability makes it an ideal choice for enterprises needing collaborative data-driven decision-making.

Security and Compliance



Snowflake prioritizes data security. With **role-based access control (RBAC)** and **end-to-end encryption**, it meets the stringent requirements of SOC 2, HIPAA, and other compliance standards. This ensures a secure environment for processing sensitive data.

Performance and Scalability



Query performance optimization is a core strength of Snowflake. Its auto-scaling capability dynamically adjusts resources based on workload, ensuring optimal performance under varying conditions. It's adept at handling large datasets and concurrent queries efficiently.



Sample Code for Data Types

Snowflake simplifies data loading with its COPY command. For JSON data:

```
COPY INTO my_table FROM  
@my_stage FILE_FORMAT = (TYPE =  
'JSON');
```

Data Integration



Snowflake integrates seamlessly with BI tools like [Tableau](#), and [Power BI](#). Its comprehensive [connectors](#) and [APIs](#) facilitate smooth integration with third-party data services, enabling a diverse set of data integration patterns.

Code Examples

Same OLD SQL



--Select all columns from a table

```
SELECT * FROM your_table;
```

--Select specific columns and filter rows

```
SELECT column1, column2 FROM your_table WHERE  
column3 > 100 ORDER BY column2 DESC;
```

--Calculate average and count of a column

```
SELECT AVG(salary) AS average_salary, COUNT(*) AS  
employee_count FROM employee_table;
```

--Inner Join between two tables

```
SELECT orders.order_id, orders.order_date,  
customers.customer_name FROM orders INNER JOIN  
customers ON orders.customer_id = customers.customer_id;
```

Code Examples

Same OLD SQL



```
-- Subquery to find employees with highest salary  
SELECT employee_name, salary FROM employees WHERE  
salary = (SELECT MAX(salary) FROM employees);
```

```
-- Rank employees based on salary within each department  
SELECT employee_name, salary, department, RANK() OVER  
(PARTITION BY department ORDER BY salary DESC) AS  
salary_rank FROM employees;
```

Code Examples

Same OLD SQL



--Create a procedure to update employee salary

```
CREATE OR REPLACE PROCEDURE update_salary(employee_id INT, new_salary INT)
```

```
RETURNS STRING
```

```
LANGUAGE JAVASCRIPT
```

```
EXECUTE AS CALLER
```

```
AS '
```

```
var sql_command = "UPDATE employees SET salary = " + new_salary + " WHERE  
employee_id = " + employee_id;
```

```
var statement1 = snowflake.createStatement({sqlText: sql_command});
```

```
statement1.execute(); return "Salary updated successfully";
```

```
';
```

--Call the procedure

```
CALL update_salary(101, 75000);
```

Code Examples



--Create a stream on a table

```
CREATE OR REPLACE STREAM your_table_stream ON TABLE your_table; -- Insert
```

--into the table and capture changes in the stream

```
--INSERT INTO your_table VALUES (1, 'John'), (2, 'Alice');
```

--Retrieve changes from the stream

```
SELECT * FROM your_table_stream;
```

--Share a database with another account

```
CREATE SHARE my_share;
```

```
GRANT USAGE ON DATABASE your_database TO SHARE my_share;
```

Getting started



Create a Free account on
<https://www.snowflake.com/en/> (30 days trial)

Provide your information.
using Georgebrown.ca email address
edition->standard
Cloud provider -> Microsoft Azure
Region ->Canada Central
Check your email to activate
supply username/password

Getting started



<https://app.snowflake.com/>

use the same username/password

Then ...

wait for a welcome email from snowflake

there will be a dedicated Login URL to click on. Bookmark it

SnowSQL (CLI client)



<https://docs.snowflake.com/en/user-guide/snowsql>

SnowSQL is the command line client for connecting to Snowflake to execute SQL queries and perform all DDL and DML operations, including loading data into and unloading data out of database tables.

<https://docs.snowflake.com/en/user-guide/snowsql-install-config>

First Tutorial Snowflake in 20 minutes



Prerequisites

1. Install SnowSQL
2. Download Sample Data, unzip it
3. Log in as the user you created for the 30 days free account (see slide on getting started with snowSQL)

 employees01

Microsoft Excel Comma S...

 employees02

Microsoft Excel Comma S...

 employees03

Microsoft Excel Comma S...

 employees04

Microsoft Excel Comma S...

 employees05

Microsoft Excel Comma S...

First Tutorial Continued



Next 3 steps

1. Create a database
2. Create a table
3. Create a Virtual Warehouse

```
CREATE OR REPLACE DATABASE sf_tuts;

SELECT CURRENT_DATABASE(), CURRENT_SCHEMA();

CREATE OR REPLACE TABLE emp_basic (
  first_name STRING ,
  last_name STRING ,
  email STRING ,
  streetaddress STRING ,
  city STRING ,
  start_date DATE
);

CREATE OR REPLACE WAREHOUSE sf_tuts_wh WITH
  WAREHOUSE_SIZE='X-SMALL'
  AUTO_SUSPEND = 180
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED=TRUE;

SELECT CURRENT_WAREHOUSE();
```

First Tutorial Continued



Next 3 steps

Stage → Copy → Query

- Internal Stages
 - Use to store data files internally within snowflake.
 - Each user and table gets an internal stage by default for staging data files
- External Stages
 - Use to store datafiles externally in [AWS](#), [GCP](#) or [Azure](#)
 - If data is already stored in those providers, then you can use the external stage to load data in Snowflake tables

```
--internal stage (copy from local machine to snowflake)
PUT file://C:\GBC\courses\CSI\sampdata\snowflakes\employees*.csv @sf_tuts.public.%emp_basic;

LIST @sf_tuts.public.%emp_basic;
```

First Tutorial Continued



Copy data into target tables

at this stage, you need to have a virtual warehouse

```
COPY INTO emp_basic  
FROM @%emp_basic  
FILE_FORMAT = (type = csv field_optionally_enclosed_by='')  
PATTERN = '.*employees0[1-5].csv.gz'  
ON_ERROR = 'skip_file';
```

- The **From** clause specifies the internal/external stage
- **File_Format** is CSV
- **Pattern** clause uses the file names matching filter
- **On_Error** specifies what to do when copy commands encounter errors

First Tutorial Continued



Query

Same old SQL query here... enjoy your SQL skills if you have or learn if you haven't

```
SELECT * FROM emp_basic;
```

```
INSERT INTO emp_basic VALUES
```

```
('Clementine','Adamou','cadamou@sf_tuts.com','10510 Sachs Road','Klenak','2017-9-22') ,  
('Marlowe','De Anesy','madamouc@sf_tuts.co.uk','36768 Northfield Plaza','Fangshan','2017-1-26');
```

```
SELECT email FROM emp_basic WHERE email LIKE '%.uk';
```

Tutorial 2 – Semi Structured Data



Typically, hierarchical data has been imported into a **VARIANT** from one of the following supported data formats

- JSON
- Avro
- ORC
- Parquet

```
| SRC |
|-----|
| {   |
|   "customer": [ |
|     { |
|       "address": "San Francisco, CA", |
|       "name": "Joyce Ridgely", |
|       "phone": "16504378889" |
|     } |
|   ], |
|   "date": "2017-04-28", |
|   "dealership": "Valley View Auto Sales", |
|   "salesperson": { |
|     "id": "55", |
|     "name": "Frank Beasley" |
|   }, |
|   "vehicle": [ |
|     { |
|       "extras": [ |
|         "ext warranty", |
|         "paint protection" |
|       ], |
|       "make": "Honda", |
|       "model": "Civic", |
|       "price": "20275", |
|       "year": "2017" |
|     } |
|   ] |
| }   |
```

Tutorial 2 – Semi Structured Data



```
CREATE OR REPLACE TABLE car_sales
(
  src variant
)
AS
SELECT PARSE_JSON(column1) AS src
FROM VALUES
({
  "date" : "2017-04-28",
  "dealership" : "Valley View Auto Sales",
  "salesperson" : {
    "id": "55",
    "name": "Frank Beasley"
  },
  "customer" : [
    {"name": "Joyce Ridgely", "phone": "16504378889", "address": "San Francisco, CA"}
  ],
  "vehicle" : [
    {"make": "Honda", "model": "Civic", "year": "2017", "price": "20275", "extras":["ext warranty", "paint protection"]}
  ]
}),
({
  "date" : "2017-04-28",
  "dealership" : "Tindel Toyota",
  "salesperson" : {
    "id": "274",
    "name": "Greg Northrup"
  },
  "customer" : [
    {"name": "Bradley Greenbloom", "phone": "12127593751", "address": "New York, NY"}
  ],
  "vehicle" : [
    {"make": "Toyota", "model": "Camry", "year": "2017", "price": "23500", "extras":["ext warranty", "rust proofing", "fabric protection"]}
  ]
}) v;
```

```
src['salesperson']['name']  
SELECT src['salesperson']['name']
```

Tutorial 2 – Semi Structured Data



Query Semi Structured Data

```
SELECT * FROM car_sales;  
  
-- Traversing Semi-structured Data  
SELECT src:dealership FROM car_sales ORDER BY 1;  
  
-- Dot Notion  
SELECT src:salesperson.name  
      FROM car_sales  
      ORDER BY 1;  
  
-- Bracket Notation  
SELECT src['salesperson']['name']  
      FROM car_sales  
      ORDER BY 1;
```

Tutorial 2 – Semi Structured Data



Query Semi Structured Data

```
SELECT * FROM car_sales;
```

-- Retrieving a Single Instance of a Repeating Element

```
SELECT src:customer[0].name, src:vehicle[0]  
FROM car_sales  
ORDER BY 1;
```

```
SELECT src:customer[0].name, src:vehicle[0].price  
FROM car_sales  
ORDER BY 1;
```

Please follow more example from the tutorial